

DIY Simulator H-Shifter

Simple to build H-shifter for PC

Jason Winfield



Introduction

Welcome to the comprehensive guide on constructing your own 3D-printed H-shifter for sim racing! This step-by-step guide combines 3D printing technology with everyday hardware to create a fully functional and customizable H-shifter. Utilizing a joystick from an Xbox controller for movement detection, coupled with an Arduino for seamless integration, this guide walks you through the entire process—from assembly to programming and tuning. Whether you're a racing enthusiast or a DIY enthusiast, embark on this journey to craft your personalized sim racing H-shifter and elevate your virtual racing experience to new heights!

This download should contain all the 3D printer files required to assemble your H-shifter along with sketch files and a itemised list of materials.

The choice of plastic used is completely up to you. I have used PET-G, which has proven strong enough to endure the rigors of sim racing.

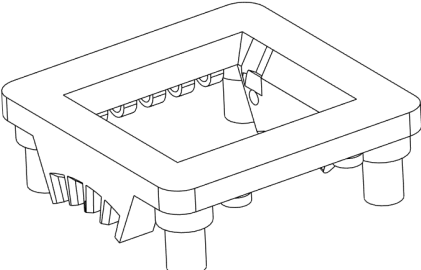
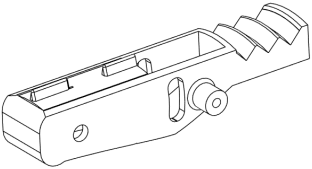
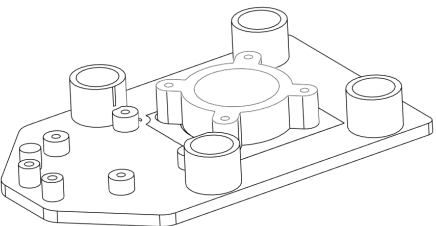
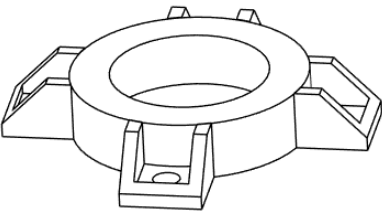
No prior programming knowledge is required to get this H-shifter up and running, as this guide will step you through all the necessary procedures.

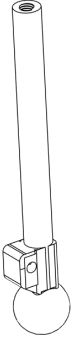
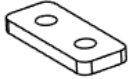
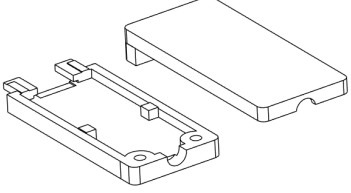
You will need some tools that you would usually find in any workshop, such as spanners and screwdrivers. A soldering iron is also required.

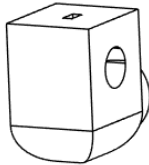
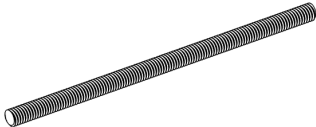
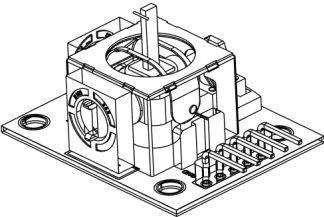
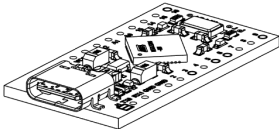
Collect your parts together and let's get on with it!

The following is required for the project. The STL files, which should have been supplied with this guide, are for use on a 3D printer. I used an FDM printer (Ender 3 V2), which is quite old, so any modern printer should not have an issue printing these files. I used PETG, but PLA should also be fine as there isn't too much stress on each component other than the shaft of the gearstick, which should be reinforced with threaded rod.

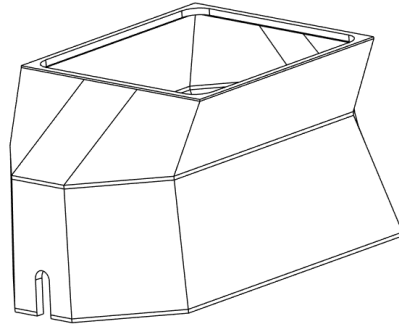
The cover is not really required, but it is supplied. It is quite a long print, so you may not consider it necessary. We will be referring to the names of the items below throughout the guide.

Top 3D 3D Printed	
Linkage 3D printed	
Base 3D printed	
Lever retainer 3D Printed	

Gear lever 3D printed	
Cable retainer 3D printed	
Alloy tubes x 4 10mm diameter 40mm long ID 8mm	
Controller board cover 3D printed	

Joystick linkage 3D printed	
Threaded rod 6mmx120mm1 Threaded rod 6mmx60mm x 4	
HW-504 Arduino joystick module	
Springs 10mmx4.5mm with loop ends	
Arduino pro Micro	

Cover 3D printed (Optional)



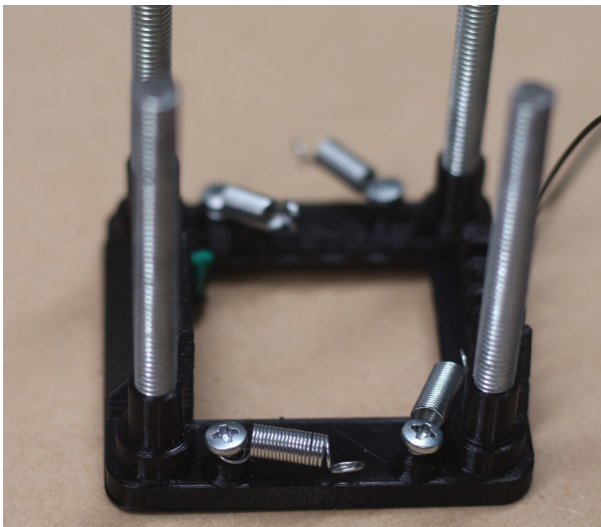
Misc:
Screws self tapping 3mmx10mm x 18
Screws 4mmx20mm x 2
Nuts 6mm x 4
Cable 5 core approx 200mm
Cable single core approx 200mm

Assembly

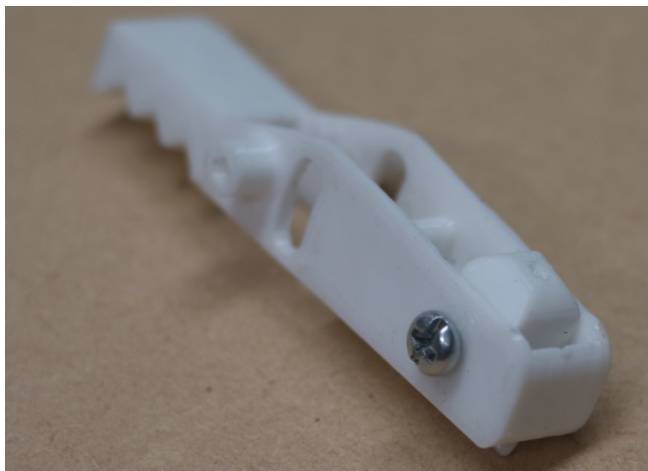
Screw the four 60mm threaded rods into each position on the top of the shifter. Be careful not to over-tighten the rods, as this could separate the layers of the 3D print. It is not necessary for the rods to be excessively tight.



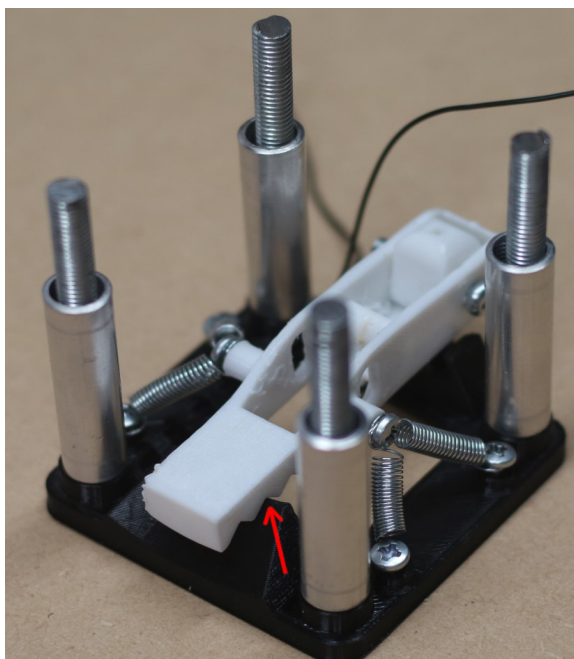
Screw the four springs into the provided locations on the top. The springs will have a flat side at the bottom. Screw them in so they sit flush (see photo). Repeat this step for all four springs



Insert the joystick linkage into the linkage arm. Insert the 4mm x 20 screw to keep it in place. There is no need to add a nut as the screw should thread into the joystick linkage and float in the linkage arm itself.



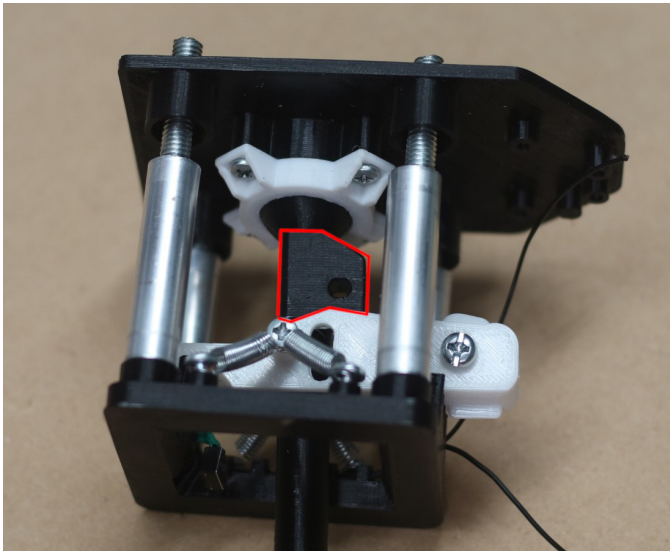
Place the linkage arm in the top assembly and screw both springs from each side into the provided holes. Note that the jagged end of the arm should face towards the rib. This configuration holds the gear shifter in place. (See diagram for reference)



Thread the 6mm x 120mm rod into the gearstick shaft. Again, be careful not to overtighten, as it could split the part. The rod should extend halfway through the ball at the bottom.

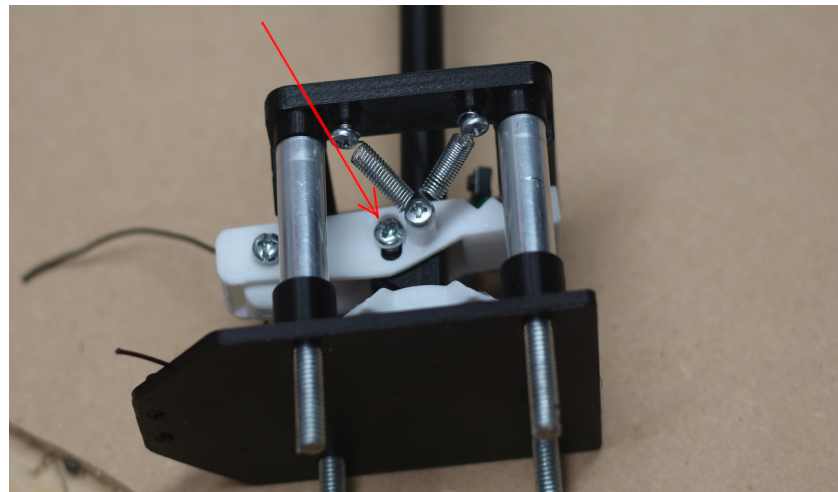
Slide the retainer over the top of the shaft and then screw the retainer in place. The gear stick should move freely. There might be a bit of play, but this won't affect the operation of the shifter.





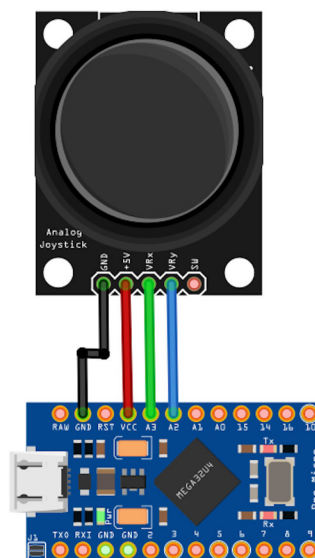
Slide the gear stick into the linkage arm, paying attention to the orientation as shown in the picture. Essentially, the hole should be facing towards the joystick linkage.

Screw the gear stick in place by adding another 4mmx20mm screw through the hole in the linkage and the gear stick

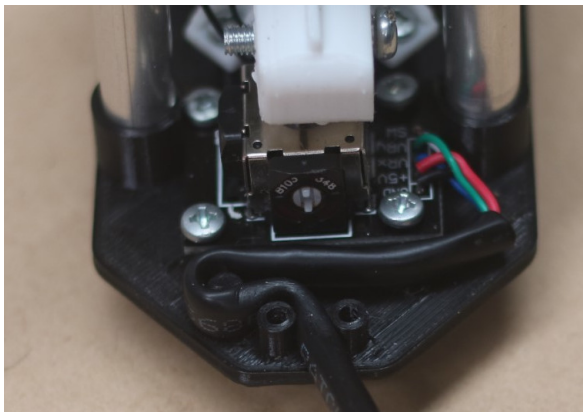


We now need to prepare the wiring. In this project, I have used an Arduino Pro Micro. You could also use an Arduino Leonardo. These controllers have specific USB functions not available in Arduino UNO and other Arduino boards so make sure you use the correct model of Arduino.

The wiring is straightforward to follow. Since the joystick is remote, the button is not utilized. While the colors are not crucial, ensuring that the correct output is connected to the corresponding input is essential

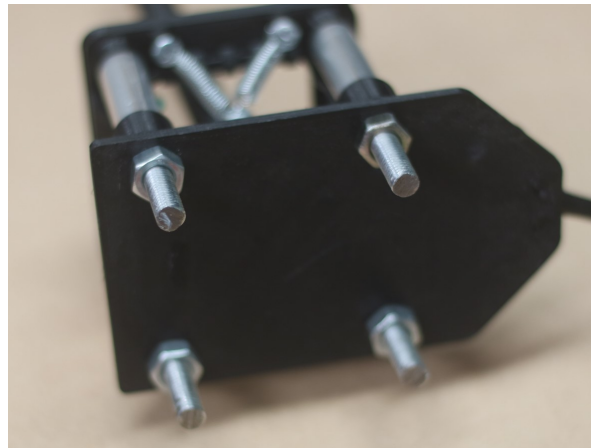


The joystick assembly should fit snugly at the front of the assembly. Raise the upper part of the assembly slightly and slot the shaft of the sensor into the hole in the joystick linkage. Screw the sensor down in place, then insert a 3mmx20mm screw through the supplied hole to hold the linkage in place. The wires from the reverse switch should already be soldered in place at this point



Thread the loom through the retaining post and then out the front of the unit. This should take any tension on the cable. Screw the wire clamp in place to keep the wires in location.

Add the 6mm nuts to the base of the unit and tighten them finger tight. Test the shifter to ensure it moves freely and stays in each gear (1 through 8). Check for any binding.



Finally, add the gear knob to the top of the shifter.



Now that our gearstick is complete, we need to connect it to our computer. Between the gearstick and the computer, we require a piece of hardware that interprets the signal from the gearstick into a format understood by your computer. To achieve this, we will utilize an Arduino. Several libraries are available for this purpose.

Arduino Joystick Library:

The Arduino Joystick Library allows you to turn your Arduino board into a USB joystick, making it compatible with computer games that support joystick input. This library is versatile and can be adapted for use in gearstick designs.

MMJoy2:

MMJoy2 is firmware specifically designed for building joystick devices using Arduino microcontrollers. It supports various input devices, including gearsticks. MMJoy2 provides a configuration software that allows you to customize the joystick's behavior.

Leo Bodnar BU0836X Joystick Board Firmware:

While not strictly Arduino-based, the Leo Bodnar BU0836X joystick board is a USB joystick controller that can be programmed with custom firmware. It's commonly used in flight simulator and racing simulator setups.

DIY USB HID Joystick Device:

Many enthusiasts have shared Arduino code for creating custom USB Human Interface Device (HID) joystick devices. This approach allows you to define the gearstick behavior and integrate it into games that support joystick input.

For the rest of the guide, we will be using the Arduino Joystick Library. Check the appendix for a link to the download.

Programming the controller

We will be using the Arduino IDE to program our controller. You will first need to download and install the Arduino joystick library, the instructions are in the link in the appendix. Once you have the IDE setup with the library copy the following code into the IDE, plugin in your controller and upload the code.

```
#include <Joystick.h>

// Define the analog pins connected to the potentiometers
#define POTENTIOMETER_PIN_X A2
#define POTENTIOMETER_PIN_Y A3

// Create an instance of the Joystick class
Joystick_ joystick;

void setup() {
  // Initialize serial communication
  Serial.begin(9600);

  // Initialize the joystick with 0 buttons
  joystick.begin(0);
}

void loop() {
  // Read the analog values from the potentiometers
  int potXValue = analogRead(POTENTIOMETER_PIN_X);
  int potYValue = analogRead(POTENTIOMETER_PIN_Y);

  // Print the current X and Y coordinates
  Serial.print("X: ");
  Serial.print(potXValue);
  Serial.print("\tY: ");
  Serial.println(potYValue);

  // Delay for stability
  delay(100); // Adjust delay as needed
}
```

Open the serial console (Ctrl-Shift-M) and observe the output. If everything is wired and configured correctly, you should see the values displayed on the screen fluctuate as you move the gear shifter.

Move the shifter to each of the eight positions. At each position, take note of the coordinates; we will need these to configure the controller. Record both the X and Y locations for gears 1 through 8.

X:0, Y:0

1	2	5	7
N	N	N	N
2	4	6	8

X:1024, Y:1024

Instructions:

When the gear stick is in position 1, the coordinates will be close to 0,0. In position 8, the coordinates will be close to 1024,1024. Moving the shifter between each gear will produce different X and Y coordinates in the Arduino console.

It's not an issue if the numbers are not perfectly evenly spread across the range, as there may be slight differences in the hardware. By the end of the process, you should have 8 pairs of numbers.

After collecting the 8 pairs of coordinates, we need to replace the code in the Arduino with the code that will run when using the gear shifter. The previous code was just for collecting the coordinates.

Please copy and paste the code below into your Arduino IDE. However, do not upload the sketch yet, as we need to make a few changes

```
#include <Joystick.h>

// Define the analog pins connected to the potentiometers
#define POTENTIOMETER_PIN_X A2
#define POTENTIOMETER_PIN_Y A3

const int positionsX[8] = {0, 0, 494, 335, 805, 570, 1023, 1023}; // Example X
coordinates for each gear position
const int positionsY[8] = {0, 1021, 0, 1023, 0, 1023, 0, 1023}; // Example Y
coordinates for each gear position
const float threshold = 0.1; // Define threshold as 10%
const int maxCoordinateValue = 1024; // Maximum possible coordinate value

Joystick_ joystick;
int previousGear = -1; // Variable to store the previous gear selection

void setup() {
  Serial.begin(9600); // Initialize serial communication
  joystick.begin(); // Initialize the joystick library
}

void loop() {
  int potXValue = analogRead(POTENTIOMETER_PIN_X); // Read X potentiometer value
  int potYValue = analogRead(POTENTIOMETER_PIN_Y); // Read Y potentiometer value
  // Calculate the maximum allowed deviation from target position
  float deviationThreshold = threshold * maxCoordinateValue;
  int currentGear = -1; // Variable to store the current gear selection
  // Check each gear position
  for (int i = 0; i < 8; i++) {
    bool withinRange = (abs(positionsX[i] - potXValue) <= deviationThreshold) &&
      (abs(positionsY[i] - potYValue) <= deviationThreshold);
    if (withinRange) {
      currentGear = i + 21; // Store the current gear selection
    }
  }
  // Check if gear selection has changed
  if (currentGear != previousGear) {
    if (currentGear != -1) {
      Serial.print("Gear ");
      Serial.print(currentGear);
      Serial.println(" selected.");
      joystick.setButton(currentGear, 1); // Set button corresponding to current
gear position
    }
    if (previousGear != -1) {
      joystick.releaseButton(previousGear); // Release button corresponding to
previous gear position
    }
    previousGear = currentGear; // Update the previous gear selection
  }
  delay(100); // Delay for stability
}
```

Calibration

You will notice the following lines in the code.

```
const int positionsX[8] = {0, 0, 494, 335, 805, 570, 1023, 1023}; // Example X
coordinates for each gear position
const int positionsY[8] = {0, 1021, 0, 1023, 0, 1023, 0, 1023};    // Example Y
coordinates for each gear position
```

Using the coordinates we discovered earlier, we need to update them in our code to match your hardware. The settings provided above are from my test unit, and while they may work, it's best to use those gathered from your earlier testing.

When gathering the coordinates, you would have written down an X and Y coordinate for each gear position. For the arrays above, we put all the X values in one array and all the Y values in the other array.

For example for 1st gear the postions is 0,0 and 2nd gear is 0,1021

```
const int positionsX[8] = {0, 0, 494, 335, 805, 570, 1023, 1023}; // Example X
coordinates for each gear position
const int positionsY[8] = {0, 1021, 0, 1023, 0, 1023, 0, 1023};    // Example Y
coordinates for each gear position
```

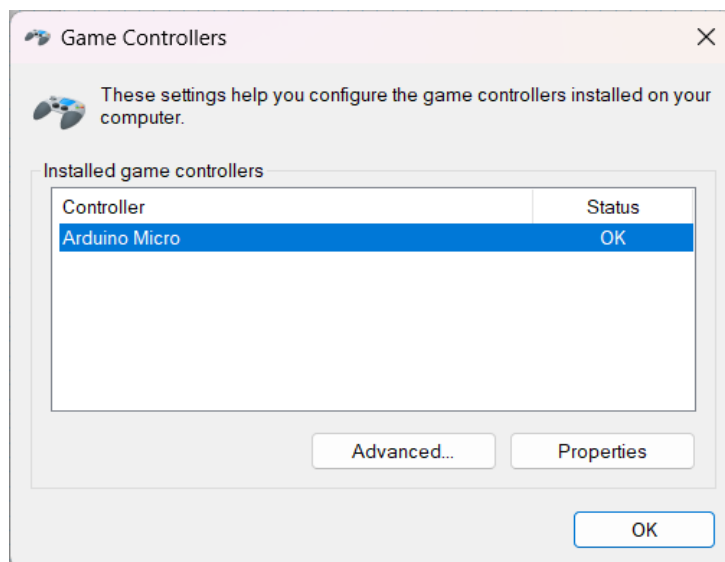
As we can see the x values go within the positionsX array and the y values go in the positionsY array. You would update all the numbers in each array. Hopefully that makes sense. If it does not please check the appendix for a link to a video which should explain it a bit better,

Once you have updated the arrays in the sketch upload it to your Arduino.

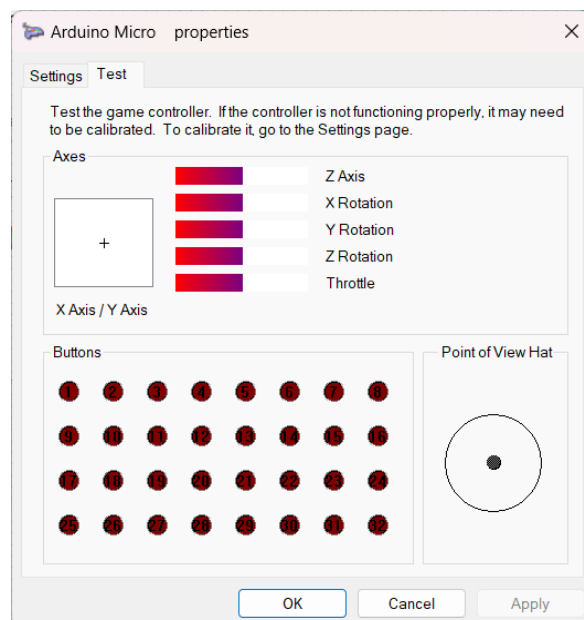
Testing

Windows has a built in game device test utility. Search for "set up USB game controllers" in Windows search.

This should provide the following dialog.



Click on Properties when Arduino Micro is highlighted, this should provide the following dialog.



As you go through each gear numbers 22 to 29 should light individually in the dialog.

Congratulations! You should now have a H-shifter!

Appendix

Arduino Software (use 1.8.19)

<https://www.arduino.cc/en/software>

Arduino Joystick Library

<https://github.com/MHeironimus/ArduinoJoystickLibrary>

As an Amazon affiliate I may make commission from Amazon purchases.

Spring kit I used

<https://amzn.to/3x6fLOZ>

Joystick Module

<https://amzn.to/3wQbzmr>

Arduino Pro Micro (UNO and other variants will not work)

<https://amzn.to/3x2y9so>

3D printed files

I printed the parts using PET-G but PLA should be fine. Although I didn't and it has been fine, I suggest you infill the areas around screw holes to 100% for strength.

Other guides that might interest you:



The spider dropper project is an excellent STEM project that introduces all ages and abilities to a cool Halloween (or anytime) project. In this project, you have all the required files to either 3D print or cut a wood chassis and create this project.

[Sample](#) [Buy Now](#)



This hotwire cutter is a great tool to cut foam for your projects. This guide has step-by-step instructions along with the latest STL files to get you up and running in no time.

[Sample](#)

[Buy Now](#)



Creating 3D-printed molds for repetitive use is a complex task. This guide explains in detail how to make your own 3D-printed molds for cement or even silicon casting using fusion 360.

[Sample](#)

[Buy Now](#)



Full build guide for my Sim racing pedals, includes new STLs and guide to configure the firmware.

[Sample](#)

[Buy Now](#)